Experiment No. : 3

Title: Measuring Central Tendency and variability of the Data

Batch:A1 **Roll No.:16014225023** **Experiment No.:3**

Aim: To compute and analyze the measures of central tendency (Mean, Median, Mode) and variability (Range, Variance, Standard Deviation, Interquartile Range) for single and multi-attribute datasets using statistical analysis in Python.

Resources needed: Resources needed: Python 3.x environment (Jupyter Notebook / Google Colab etc.), any statistical analysis tool

---

Theory

Mathematically, central tendency means measuring the center or distribution of location of values of a data set. It gives an idea of the average value of the data in the data set and also an indication of how widely the values are spread in the data set. That in turn helps in evaluating the chances of a new input fitting into the existing data set and hence probability of success.

Measures of Central Tendency:

There are three main measures of central tendency:

- **Mean:** It is the average value of the data, which is a division of the sum of the values by the number of values.
- **Median:** It is the middle value in a distribution when the values are arranged in ascending or descending order.
- **Mode:** It is the most commonly occurring value in a distribution.

Measures of Variability:

Variability describes how far apart data points lie from each other and from the center of a distribution. Along with measures of central tendency, measures of variability give you descriptive statistics that summarize your data. Variability is also referred to as spread, scatter, or dispersion. It is most commonly measured with the following:

- **Range:** The difference between the highest and lowest values.
- **Interquartile Range (IQR):** The range of the middle half of a distribution.
- **Standard Deviation:** Average distance from the mean.
- **Variance:** Average of squared distances from the mean.
- **Coefficient of Variation:** Ratio of the standard deviation to the mean.

Activity:

1. Analyze the data to find out frequency, mean, mode, median, standard deviation, variance, and interquartile range of the data.
2. Write a programming code with a procedure to compute mean, mode, median, variance, standard deviation, and interquartile range of a given sample data **without using in-built functions**.
3. Compute and analyze the grouped data to find out frequency, mean, mode, median, standard deviation, variance, and interquartile range of data.

Results: (program with output)

```
import pandas as pd



df = pd.read_csv('/content/irrigation_prediction.csv')



display(df.head())
```

	Soil_Type	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rainfall_mm	Sunlight_Hours	Wind_Speed_kmh	Crop_Type	Crop_Growth_Stage	Season	Irrigation_Type	Water_Source	Field_Area_hectare	Mulching_Used	Previous_Irrigation_mm	Region	Irrigation_Need
0	Clay	8.14	38.48	0.42	2.17	21.90	31.19	1167.70	4.01	1.97	Wheat	Vegetative	Rabi	Rainfed	Reservoir	4.73	Yes	1.98	South	Low
1	Silt	8.41	50.98	0.38	0.23	36.50	28.01	831.28	10.72	18.82	Maize	Flowering	Zaid	Canal	Groundwater	12.22	Yes	33.56	Central	Medium
2	Sandy	7.71	40.07	1.09	2.18	41.83	76.41	1844.45	7.75	19.03	Cotton	Harvest	Rabi	Drip	Reservoir	5.52	Yes	34.62	South	Low
3	Clay	5.96	12.75	1.56	0.40	37.22	43.32	306.26	8.90	11.44	Wheat	Sowing	Kharif	Canal	Reservoir	1.43	Yes	84.03	North	Medium
4	Clay	7.76	18.58	0.95	2.52	22.38	86.44	1875.83	10.39	11.28	Cotton	Sowing	Zaid	Canal	River	2.52	No	60.86	South	Medium

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 20 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Soil_Type                 10000 non-null  object
 1   Soil_pH                   10000 non-null  float64
 2   Soil_Moisture             10000 non-null  float64
 3   Organic_Carbon            10000 non-null  float64
 4   Electrical_Conductivity   10000 non-null  float64
 5   Temperature_C             10000 non-null  float64
 6   Humidity                  10000 non-null  float64
 7   Rainfall_mm               10000 non-null  float64
 8   Sunlight_Hours            10000 non-null  float64
 9   Wind_Speed_kmh            10000 non-null  float64
 10  Crop_Type                 10000 non-null  object
 11  Crop_Growth_Stage         10000 non-null  object
 12  Season                    10000 non-null  object
 13  Irrigation_Type           10000 non-null  object
 14  Water_Source              10000 non-null  object
 15  Field_Area_hectare        10000 non-null  float64
 16  Mulching_Used             10000 non-null  object
 17  Previous_Irrigation_mm    10000 non-null  float64
 18  Region                    10000 non-null  object
 19  Irrigation_Need           10000 non-null  object
dtypes: float64(11), object(9)
memory usage: 1.5+ MB
```

```
import pandas as pd



df = pd.read_csv('/content/irrigation_prediction.csv')



num_cols = [

    'Soil_pH',
```

```
    'Soil_Moisture',
    'Temperature_C',
    'Humidity',
    'Rainfall_mm',
]


summary_list = []


for col in num_cols:
    data = df[col].dropna().tolist()
    n = len(data)


    total_sum = 0
    for val in data:
        total_sum += val
    mean = total_sum / n


    sorted_data = sorted(data)
    mid = n // 2
    if n % 2 == 0:
        median = (sorted_data[mid - 1] + sorted_data[mid]) / 2.0
    else:
```

```
        median = float(sorted_data[mid])


    freq_dict = {}
    for val in data:
        rounded_val = round(val, 2)
        if rounded_val in freq_dict:
            freq_dict[rounded_val] += 1
        else:
            freq_dict[rounded_val] = 1


    max_freq = 0
    for count in freq_dict.values():
        if count > max_freq:
            max_freq = count


    mode = None
    for k, v in freq_dict.items():
        if v == max_freq:
            mode = k
            break


    rng = max(data) - min(data)
```

```
if n % 2 == 0:
    lower_half = sorted_data[:mid]
    upper_half = sorted_data[mid:]
else:
    lower_half = sorted_data[:mid]
    upper_half = sorted_data[mid + 1:]


m1 = len(lower_half) // 2
if len(lower_half) % 2 == 0:
    q1 = (lower_half[m1 - 1] + lower_half[m1]) / 2.0
else:
    q1 = float(lower_half[m1])


m3 = len(upper_half) // 2
if len(upper_half) % 2 == 0:
    q3 = (upper_half[m3 - 1] + upper_half[m3]) / 2.0
else:
    q3 = float(upper_half[m3])


iqr = q3 - q1
```

```
    sq_diff_sum = 0
    for x in data:
        sq_diff_sum += (x - mean) ** 2
    variance = sq_diff_sum / (n - 1)
    std_dev = variance ** 0.5


    summary_list.append({
        'Attribute': col,
        'Mean': round(mean, 2),
        'Median': round(median, 2),
        'Mode': mode,
        'Range': round(rng, 2),
        'Q1': round(q1, 2),
        'Q3': round(q3, 2),
        'IQR': round(iqr, 2),
        'Variance': round(variance, 2),
        'Std Dev': round(std_dev, 2),
    })


df_activity1 = pd.DataFrame(summary_list)
print('ACTIVITY 1: MULTI-ATTRIBUTE STATISTICAL ANALYSIS')
display(df_activity1)
```

ACTIVITY 1: MULTI-ATTRIBUTE STATISTICAL ANALYSIS

	Attribute	Mean	Median	Mode	Range	Q1	Q3	IQR	Variance	Std Dev
0	Soil_pH	6.49	6.47	5.66	3.40	5.64	7.35	1.71	0.96	0.98
1	Soil_Moisture	36.97	37.24	52.38	57.00	22.86	50.94	28.08	269.97	16.43
2	Temperature_C	26.99	27.09	32.74	30.00	19.46	34.50	15.04	75.07	8.66
3	Humidity	60.08	60.04	48.24	70.00	42.85	77.71	34.86	407.55	20.19
4	Rainfall_mm	1252.50	1250.34	226.24	2499.31	634.12	1880.30	1246.18	512057.89	715.58

```
sample_data = [12, 15, 12, 18, 20, 22, 25, 12, 18, 30]



def compute_mean(data):

    total = 0

    for x in data:

        total += x

    return total / len(data)



def compute_median(data):

    s = sorted(data)

    n = len(s)

    mid = n // 2

    if n % 2 == 0:

        return (s[mid - 1] + s[mid]) / 2.0

    else:

        return float(s[mid])
```

```
def compute_mode(data):
    counts = {}
    for x in data:
        if x in counts:
            counts[x] += 1
        else:
            counts[x] = 1

    max_c = 0
    for count in counts.values():
        if count > max_c:
            max_c = count

    for k, v in counts.items():
        if v == max_c:
            return k

def compute_range(data):
    return max(data) - min(data)

def compute_variance(data):
    m = compute_mean(data)
```

```
    n = len(data)
    sq_diff_sum = 0
    for x in data:
        sq_diff_sum += (x - m) ** 2
    return sq_diff_sum / (n - 1)


def compute_std_dev(data):
    return compute_variance(data) ** 0.5


def compute_iqr(data):
    s = sorted(data)
    n = len(s)
    mid = n // 2


    if n % 2 == 0:
        lower = s[:mid]
        upper = s[mid:]
    else:
        lower = s[:mid]
        upper = s[mid + 1:]


    q1 = compute_median(lower)
```

```
    q3 = compute_median(upper)

    return q3 - q1, q1, q3




mean_val = compute_mean(sample_data)

median_val = compute_median(sample_data)

mode_val = compute_mode(sample_data)

range_val = compute_range(sample_data)

var_val = compute_variance(sample_data)

std_val = compute_std_dev(sample_data)

iqr_val, q1_val, q3_val = compute_iqr(sample_data)



print('ACTIVITY 2: SAMPLE DATA COMPUTATIONS')

print(f'Sample Data: {sample_data}')

print(f'Mean: {mean_val:.2f}')

print(f'Median: {median_val:.2f}')

print(f'Mode: {mode_val}')

print(f'Range: {range_val:.2f}')

print(f'Q1: {q1_val:.2f}')

print(f'Q3: {q3_val:.2f}')

print(f'IQR: {iqr_val:.2f}')

print(f'Variance: {var_val:.2f}')
```

```
print(f'Standard Deviation: {std_val:.2f}')
```

```
ACTIVITY 2: SAMPLE DATA COMPUTATIONS
Sample Data: [12, 15, 12, 18, 20, 22, 25, 12, 18, 30]
Mean: 18.40
Median: 18.00
Mode: 12
Range: 18.00
Q1: 12.00
Q3: 22.00
IQR: 10.00
Variance: 36.49
Standard Deviation: 6.04
```

```
import pandas as pd



df = pd.read_csv('/content/irrigation_prediction.csv')

grouped_attribute = df['Soil_Moisture'].dropna().tolist()



num_bins = 5

min_val = min(grouped_attribute)

max_val = max(grouped_attribute)

width = (max_val - min_val) / num_bins



classes = []

midpoints = []

for i in range(num_bins):
```

```
    low = min_val + i * width
    high = min_val + (i + 1) * width
    classes.append((low, high))
    midpoints.append((low + high) / 2)

frequencies = [0] * num_bins
for x in grouped_attribute:
    for i in range(num_bins):
        low = classes[i][0]
        high = classes[i][1]

        if i == num_bins - 1:
            if x >= low and x <= high:
                frequencies[i] += 1
        else:
            if x >= low and x < high:
                frequencies[i] += 1

N = sum(frequencies)

cf = []
cum = 0
```

```
for f in frequencies:
    cum += f
    cf.append(cum)

sum_fm = 0
for i in range(num_bins):
    sum_fm += frequencies[i] * midpoints[i]
grp_mean = sum_fm / N

def interpolate_percentile(p):
    target = p * N
    idx = 0

    for i in range(len(cf)):
        if cf[i] >= target:
            idx = i
            break

    L = classes[idx][0]
    if idx > 0:
        prev_cf = cf[idx - 1]
    else:
```

```
        prev_cf = 0


    return L + ((target - prev_cf) / frequencies[idx]) * width


grp_median = interpolate_percentile(0.5)
grp_q1 = interpolate_percentile(0.25)
grp_q3 = interpolate_percentile(0.75)
grp_iqr = grp_q3 - grp_q1


max_f = max(frequencies)
modal_idx = frequencies.index(max_f)
L_mode = classes[modal_idx][0]
f1 = frequencies[modal_idx]


if modal_idx > 0:
    f0 = frequencies[modal_idx - 1]
else:
    f0 = 0


if modal_idx < len(frequencies) - 1:
    f2 = frequencies[modal_idx + 1]
else:
```

```
    f2 = 0


denom_mode = 2 * f1 - f0 - f2
if denom_mode != 0:
    grp_mode = L_mode + ((f1 - f0) / denom_mode) * width
else:
    grp_mode = midpoints[modal_idx]


var_sum = 0
for i in range(num_bins):
    var_sum += frequencies[i] * ((midpoints[i] - grp_mean) ** 2)
grp_var = var_sum / (N - 1)
grp_std = grp_var ** 0.5


freq_table_data = []
for i in range(num_bins):
    if i < num_bins - 1:
        interval_str = f'[{classes[i][0]:.2f} - {classes[i][1]:.2f})'
    else:
        interval_str = f'[{classes[i][0]:.2f} - {classes[i][1]:.2f}]'


    freq_table_data.append({
```

```
        'Class Interval': interval_str,
        'Midpoint (m)': round(midpoints[i], 2),
        'Frequency (f)': frequencies[i],
        'Cumulative Freq (CF)': cf[i],
    })

freq_df = pd.DataFrame(freq_table_data)

grouped_stats_summary = pd.DataFrame([{
    'Grouped Mean': round(grp_mean, 2),
    'Grouped Median': round(grp_median, 2),
    'Grouped Mode': round(grp_mode, 2),
    'Grouped Q1': round(grp_q1, 2),
    'Grouped Q3': round(grp_q3, 2),
    'Grouped IQR': round(grp_iqr, 2),
    'Grouped Variance': round(grp_var, 2),
    'Grouped Std Dev': round(grp_std, 2),
}])

print('ACTIVITY 3: GROUPED DATA FREQUENCY TABLE')
display(freq_df)
```

```
print('\nACTIVITY 3: GROUPED DATA STATISTICAL MEASURES')

display(grouped_stats_summary)
```

```
ACTIVITY 3: GROUPED DATA FREQUENCY TABLE
```

	Class Interval	Midpoint (m)	Frequency (f)	Cumulative Freq (CF)
0	[8.00 - 19.40)	13.7	1911	1911
1	[19.40 - 30.80)	25.1	1949	3860
2	[30.80 - 42.20)	36.5	2010	5870
3	[42.20 - 53.60)	47.9	2106	7976
4	[53.60 - 65.00]	59.3	2024	10000

```
ACTIVITY 3: GROUPED DATA STATISTICAL MEASURES
```

	Grouped Mean	Grouped Median	Grouped Mode	Grouped Q1	Grouped Q3	Grouped IQR	Grouped Variance	Grouped Std Dev
0	36.94	37.27	48.35	22.85	51.02	28.18	257.09	16.03

Post-Lab Questions:

1. **What are the various applications of central tendency and variability of data?**

Applications of Central Tendency and Variability

- **Precision Agriculture & Smart Farming:** Central tendencies (mean/median soil moisture or pH) establish optimal baselines for automated irrigation triggers, while variability metrics (standard deviation of temperature or rainfall) indicate environmental stability and crop risk.
- **Financial & Risk Analysis:** Mean portfolio return measures expected performance, whereas variance and standard deviation quantify market volatility and investment risk.
- **Manufacturing Quality Control:** Assembly lines use the target mean/mode for item dimensions and enforce tolerance limits using Six Sigma thresholds based on standard deviation.
- **Socioeconomic & Public Policy:** Government agencies rely on median income rather than mean income to evaluate wealth distribution without distortion from extreme high earners.
- **Anomaly & Outlier Detection:** Data scientists utilize the Interquartile Range ($1.5 \times \text{IQR}$ rule) to isolate sensor glitches, fraud, or noisy data points before training machine learning models.

2. **What are the limitations of Central Tendency and variability of Data?**

Limitations of Central Tendency

- **Outlier Sensitivity:** The arithmetic mean is heavily distorted by extreme values (e.g., a few massive rainfall spikes disproportionately inflate average rainfall).
- **Loss of Distribution Geometry:** Summary metrics hide distribution shape; a bimodal distribution with two distinct peaks yields a single central value that represents neither group accurately.
- **Mode Instability in Continuous Data:** Continuous floating-point features rarely repeat exact values, rendering the unbinned mode uninformative or multimodal.
- **Non-Numeric Inapplicability:** Mean and median cannot be computed on nominal or categorical attributes (e.g., Soil_Type or Crop_Type).

Limitations of Variability

- **Range Boundary Dependency:** Range relies exclusively on the minimum and maximum values, making it entirely sensitive to extreme outliers while ignoring internal spread.
- **Squared Distance Amplification:** Variance squares individual deviations ($(x_i - \bar{x})^2$), severely over-penalizing extreme values and yielding squared units (mm^2) that lack direct physical meaning.
- **IQR Tail Data Erasure:** While resistant to outliers, the Interquartile Range discards the top and bottom 25% of data, ignoring tail risks and extreme edge events.
- **Assumption of Symmetry:** Standard deviation assumes a roughly normal, symmetric distribution; applying it to heavily skewed data provides a misleading measure of spread.

Course Outcomes: (refer exp list)

- **CO1:** Apply fundamental concepts of descriptive statistics—including central tendency and variability—to analyze continuous single and multi-attribute datasets.
- **CO2:** Develop algorithmic procedures in Python to compute statistical parameters (mean, median, mode, range, variance, standard deviation, and IQR) without relying on built-in statistical modules.
- **CO3:** Construct class interval frequency tables and execute continuous data binning to evaluate grouped statistics using linear interpolation formulas.
- **CO4:** Analyze data dispersion, symmetry, and spread to evaluate dataset behavior for data preprocessing and predictive machine learning models.

Conclusion:

In this experiment, measures of central tendency and variability were successfully computed and analyzed for both multi-attribute ungrouped data and grouped continuous data from the irrigation_prediction.csv dataset ($N = 10,000$). Custom Python procedures written from first principles verified key statistical properties across environmental attributes such as Soil_pH, Soil_Moisture, Temperature_C, Humidity, and Rainfall_mm.

The results demonstrated high mathematical fidelity between raw ungrouped metrics and 5-bin grouped calculations (e.g., Soil_Moisture raw mean of 36.97 vs. grouped mean of 36.94). Overall, this lab successfully provided hands-on experience in programmatic statistical computation, frequency table construction, continuous binning interpolation, and descriptive data interpretation.

Grade: AA / AB / BB / BC / CC / CD /DD
Signature of faculty in-charge with date

References:
Books/ Journals/ Websites:

1. S.C. Gupta, V. K. Kapoor, *Fundamentals of Mathematical Statistics*, Sultan Chand and Sons, 2020.
2. P. N. Tan, M. Steinbach, Vipin Kumar, *Introduction to Data Mining*, Pearson Education, 2016.
3. Han, Kamber, *Data Mining Concepts and Techniques*, Morgan Kaufmann, 3rd Edition, 2012
4. NumPy Documentation - https://numpy.org/doc/
5. Pandas Documentation - https://pandas.pydata.org/